

Forensic Volatility 3 Memory Analysis Cheat Sheet

Plugin Name	Description	Forensic Significance
<i>windows.registry.hivelist</i>	Enumerates are registry hives (HKEY_CLASSES_ROOT, HKEY_CURRENT_USER, HKEY_LOCAL_MACHINE, HKEY_USERS, and HKEY_CURRENT_CONFIG), including file path location and state of the file.	Information gathered will be useful for further registry analysis as it provides the memory offset of the file, that can be used in memory mapping but also the file path of the hive for ease of access when later analysing the memory capture through software such as FTKImager or Autopsy.
<i>windows.info</i>	Provides basic information about the Operating System (OS). This information includes system time, root file path, OS product type and version, and more.	Information provides a good starting point for further investigation and allows more OS specific investigations to be conducted if the OS information was not initially known.
<i>windows.mbr</i>	Scans for Master Boot Records (MBRs) which is information required to complete the computer booting process.	This information may be of use in the case where booting of the system is faulty and analysis of the booting set up is required. If malware has corrupted the booting files, values retrieved by this plugin will have been altered.
<i>windows.verinfo</i>	Provides a dumps of virtual address descriptors information. This information includes Process IDs, process name, base and more.	Virtual memory is commonly used when processes running on the system require more memory to run than what is provided by the RAM. Virtual memory is located on both RAM and the hard drive. This means that the information retrieved virtual memory may provide deeper insight into a system as compared to only limiting analysis to the RAM.
<i>windows.cmdline</i>	The cmdline plugin extracts the command-line arguments used to launch each process.	This will help us analyze how processes were started. For example, whether any scripts or executables were launched with hidden parameters or elevated privileges. It provides clues about potential malicious scripts or PowerShell commands executed during the test.
<i>windows.cmdscan</i>	Provides the command history of the system and thus providing volatile data possibly including credentials (both local system and online accounts), network connections, malware intrusions, registry hives and more.	Some of this information is not stored on the system hard drive but provides significant evidence of processes running on the system at or before the time of memory capture.
<i>windows.consoles</i>	Used to extract the system's command history similar to the windows.cmdscan plugin but also provide the output of the commands run of the system.	Identical to windows.cmdscan with the additional benefit of view what the output of the run commands were. This provides insight into what data was accessed and how it may have been manipulated.

<i>windows.dlllist</i>	The dlllist plugin lists all dynamic-link libraries (DLLs) loaded into each process's memory space.	This output helps us identify unusual or unsigned DLLs injected into legitimate processes. Detecting these anomalies allows us to verify whether the injected payload is present and understand the infection mechanism used.
<i>windows.envvars</i>	Prints environment variables configured on the captured system. This information include the hardware components and configuration, processor's current directory, temporary directory, session name, computer name, and user names	Information gathered assists in navigating the system and understanding important configuration settings of both hardware and software. Recording custom variables may also be important in understanding the user of the system.
<i>windows.malfind</i>	The malfind plugin scans for suspicious memory regions with executable permissions or injected code.	This is a key plugin for our post-injection analysis. It allows us to locate memory sections containing injected payloads or shellcode. We can dump and review these sections to confirm whether the test injection was successful and where it resides in memory.
<i>windows.netscan</i>	The netscan plugin identifies network connections and sockets, showing details like local and remote IPs, ports, and associated process IDs.	This data will help us trace network activity related to specific processes. By reviewing unknown outbound connections or listening ports, we can determine if any suspicious network behavior was established after malware injection.
<i>windows.pslist</i>	The pslist plugin enumerates active processes captured in the Windows memory image. It displays details such as Process ID (PID), Parent Process ID (PPID), thread counts, and creation times.	This output will help us analyze all active processes at the time of capture. By comparing it with the results from psscan, we can identify any hidden or terminated processes, which may indicate process-hollowing or injected malware activity within the system.
<i>windows.psscan</i>	The psscan plugin scans the raw memory for process structures, including those that may no longer appear in the active process list.	This plugin will allow us to detect any processes that are not visible in pslist. Comparing both results will help us confirm whether any stealth or hidden processes exist in the memory image, which could be linked to injected or malicious code.
<i>windows.cachedump</i>	Dumps Local Security Authority (LSA) secrets from the capture memory dump. Information includes credentials from computer's active directory domain services, credentials for windows services, credentials for scheduled tasks credentials for websites, and credentials for any Microsoft accounts.	This plugin is of great assistance in recovering credentials that may be later required for decrypting and accessing protected information.
<i>windows.callbacks</i>	Print system-wide notification routines most commonly used to monitor particular system.	This monitoring of a system is commonly used for rootkit drivers and information provided by this plugin may indicate spyware, a backdoor, or other forms of malware.

<i>windows.filescan</i>	Outputs file pathnames of all files located on the system, including both directory files and normal files. The output also includes the memory offset of the file.	Provided information is crucial in collecting evidence of malware located on a system as navigating any possible files created or manipulated by the malware. The offset also assists in file carving if required.
<i>windows.getservicesids</i>	Calculates and displays the Service IDs (SIDs) and service names from the captured system. Service information is retrieved from the SYSTEM\CurrentControlSet\Services registry file.	SIDs may be later used in construction a timeline of active services when reading and comparing the outputs of other Volatility 3 plugins.
<i>windows.getsids</i>	Prints Security Identifiers (SIDs) running each process.	SIDs indicate the user account running the process and may indicate an preexisting account compromised by malware or an account that was created by the malware. The later indicates root access and will escalate the severity of the attack. This also includes SIDs belonging to the OS.
<i>windows.handles</i>	Prints Process IDs (PIDs), process names, memory offsets, handle values, process type, granted accesses, and source of the process (eg. System, wininit.exe, user software, etc)	Important when searching for a particular process via PID or name that may be associated with malware. Will also be of assistance in finding the process type category given to the malware and source that ran the malware.
<i>windows.hashdump</i>	Extracts and decrypts hashes from memory, including cached domain credentials.	The collected credentials can be later used in forensic analysis when accessing memory that requires a key to decrypt.
<i>windows.lsadump</i>	Identical to windows.cachedump but dumps hash data and includes Computer user passwords.	Identical forensic significance to windows.cachedump.
<i>windows.mftscan.MFTScan</i>	Scans for Master File Table (MFT) files in the captured memory image.	This table include records of all file and directory on the windows system and their metadata such as creation and last modification timestamps. Information may assist the construction of a forensic timeline.
<i>windows.modscan</i>	Scans for modules present in a memory capture. This information includes previously unloaded drivers and drivers that have been hidden by a potential rootkit, as well as file locations.	This can assist in understanding the actions of a rootkit and in the construction of a timeline for forensic analysis reporting.
<i>windows.modules</i>	Prints list of loaded modules and drivers currently on the system. This information includes name, base address (location in system), path, size, offset, and file output.	Modules may be used as a method of persistence for a malware and thus analysing the information provided by this plugin will assist in indicating the presence of a malware as well as then understanding the functionality of the malware.
<i>windows.mutantscan</i>	Displays all PIDs and thread IDs of a mutex owner if one exists, as well as displaying the names of the mutexes.	A mutex is a mutual exclusion lock and is used to protect data and resources from being accessed by two or more services at any one time. This may be of use in digital forensics to discover any locked information that requires

		a key that may able to be retrieved through other Volatility 3 plugins such as windows.hashdump.
<i>windows.privileges</i>	Lists process token privileges, including, PID, process, privilege, attribute, and description.	The recorded privilege tokens include altering permissions of files or use of privileges that are not the default. This can indicate a rootkit and privilege escalation.
<i>windows.sessions</i>	Scans the environment variables and lists processes with session information. This information includes PID, session type, process start time, and more.	Additional method of constructing a timeline of active drivers besides windows.modscan and windows.modules plugins.
<i>windows.skeleton_key_check</i>	Scans for common indicators of skeleton key malware.	Skeleton key malware is a type of malware that bypasses authentication on systems that only have single factor authentication. Scanning for this particular type of malware will result in a near definite conclusion if the type of malware is present on the system.
<i>windows.ssdt</i>	Prints the system call table.	The system call table contains pointers to functions that implement system calls, and these system calls are used by the kernel to maintain the operating system. If malware is present, altering the system calls table may be a method of persistence.
<i>windows.suspicious_threads</i>	Lists suspicious userland process threads.	Userland threads are managed by programs installed and owned by the user and are not involved in managing or the OS. This plugin allows processes that have been installed by a user or malware to be analysed. This assists in investigations involving malware that doesn't alter or manipulate the kernel.
<i>windows.symlinkscan</i>	Similar to the windows.filescan plugin but limited to printing the information of symbolic link files. Provides creation time, and file location and linked location of symbolic link.	A given malware may create symbolic links between files to simplify file navigation for an attacker to later take advantage of. An attacker may manually create these symbolic links but the by using this plugin along side others such as windows.handles, windows.consoles, or windows.cmdscan, an understanding of the attacker's intentions may be better understood.
<i>windows.thrdscan</i>	Locates ETHREAD object in physical memory. Information includes Parent PIDs (PPIDs) that can be used to identify hidden processes that may have been hidden to other Volatility 3 plugins that report PIDs.	Used when a malware attempts to obscure the recording of the processes that its running and remain undetected.
<i>windows.unloadedmodules</i>	Prints list of recently unloaded drivers that have been stored in memory for debugging purposes.	Additional method of constructing a timeline of active drivers besides windows.modscan and windows.modules plugins.

windows.vadinfo

Similar to windows.verinfo but provides extended information about virtual addresses including starting and ending virtual addresses in RAM, virtual address tags and flags, name, and permissions.

Provides a deeper understanding of the virtual memory as compared to windows.verinfo if further investigation is required.

References

<https://www.eyehatemalwares.com/digital-forensics/memory-analysis/volatility-consoles/>

<https://medium.com/@mozammalhossaintanvir/volatility3-cheat-sheet-1980cf40bff2>

<https://blog.onfvp.com/post/volatility-cheatsheet/>

<https://github.com/volatilityfoundation/volatility/wiki/command-reference>

<https://www.sangfor.com/glossary/cybersecurity/what-is-mbr-master-boot-record#:~:text=A%20master%20boot%20record%20is,of%20the%20computer's%20operating%20system.>

<https://www.ibm.com/docs/en/aix/7.1.0?topic=programming-using-mutexes>

<https://learn.microsoft.com/en-us/windows-server/security/windows-authentication/credentials-processes-in-windows-authentication>

<https://docs.oracle.com/cd/E19253-01/816-4557/auditref-23/index.html#:~:text=The%20privilege%20token%20records%20the,of%20that%20privilege%20is%20recorded.>

<https://www.secureworks.com/research/skeleton-key-malware-analysis>